Structuring large real-time systems with performance polymorphism

Kevin B. Kenny[1] Kwei–Jay Lin[2]

Department of Computer Science
University of Illinois at Urbana–Champaign
Urbana, Illinois 61801

Abstract

Sophisticated and flexible real-time systems may use several versions of a program fragment that performs a particular computation. These versions differ only in their performance parameters, such as the time required, the resources consumed, and the precision of the results. This paper presents a model of *performance polymorphism,* whereby the process of selecting a version from among the available set is automated. It discusses the binding problem for this new model of polymorphism and the problem of allocating resources among computations that all support performance polymorphism.

1 Introduction

A great amount of research in recent years has been devoted to the structuring of *hard* real-time systems: systems in which computations must be completed at appropriate times. Large-scale real-time systems are now being designed for such applications as avionics, space projects, and submarine systems. These systems require a great deal of flexibility in dealing with unforeseen circumstances. They often must be tolerant of hardware faults, reconfiguring around failed system components at run time. Moreover, they often have an expected service life exceeding that of the underlying hardware on which they are implemented, requiring that attention be paid to portability.

One approach to addressing these considerations is to implement multiple versions of a function that carries out a given computation. These versions all perform the same task, but differ only in the amount of time and resources they consume, the system configuration to which they are adapted, the precision of the results that they return, and similar performance criteria. The versions may be specialized for a particular machine architecture, a particular problem size, a particular optimization strategy, and so on. The multiple versions may be supplied by the programmer, as when different algorithms adapted to problems of different sizes are supplied. They may also be generated automatically, as when a program rewriting tool is used to adapt a sequential program to a vector or parallel machine.

In this paper, we present a model for specifying these multiple versions. We call this model *performance polymorphism,* because of its similarities to the conventional polymorphism where different functions carry out the same operation on different types of data [2].

Section 2 presents some background information, and Section 3 describes the language framework in which the model is implemented. Section 4 discusses the *binding* problem of assigning one of the available candidates to a given function invocation. Section 5 discusses the *allocation* problem of dividing the available time and resources among multiple functions, each of which has multiple implementations. Finally, Section 6 presents our experience with an implementation of this model.

2 Background and Related Work

In the design of real-time systems, we are beginning to see a proliferation of versions of programs that perform the same task. Versions arise that are specialized to a given machine architecture, for example, a conventional scalar machine versus a vector or parallel machine. Different algorithms are used on problems of different scale, for example, a fast $O(n^2)$ algorithm versus a slow $O(n \log n)$ one. Different optimization strategies may be used by the compilers, for example, saving space versus saving time. Finally, different methods may be used to return results of different precision.

Any combination of these variations may be present in a given problem, complicating the problem of having the system meet all its deadlines. Some of these variations can arise late in the development cycle, in rehosting a program to a new architecture, in adding a new optimization technique, or in addressing a particular performance deficiency by introducing a new algorithm. Moreover, the need for fault tolerance means that some of the decisions about the choice of a version may not be made until run time. A version may depend on a hardware component that has failed, and an alternative version, perhaps inferior in some other respect, must be chosen. Even in the absence of any failure, the best choice of version may depend on the structure of a problem, as there may be, for instance, versions that adapt well to large data sets but are prohibitively inefficient with small ones.

One conventional way to deal with these multiple versions is either to perform analysis in advance and select a single version that is suited to all the cases; this, obviously, may not be possible. The more sophisticated conventional technique is to choose some set of versions that cover all the cases, and select from among them with an **if**... **else if**... **else**... construct. This technique can surely select a version, but it has a number of drawbacks.

First, the construction of the conditions of the **if** statement is prone to error. Some means of determining the performance of the various versions must be incorporated, and

[1] Author's current address: Northrop Research and Technology Center, One Research Park, Palos Verdes Peninsula, CA 90732

[2] This research was sponsored by the National Aeronautics and Space Administration under grant NASA-NAG-1-613, by the Office of Naval Research under grant N00014-89-J-1181, and by the National Science Foundation under grant CCR-89-11773.

CH2933-0/90/0000/0238$01.00 © 1990 IEEE

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

the condition that 'version A is better than version B' must be derived. If the performance of a version changes, not only may the conditions change, but the structure of the **if** statement may change as well—a sufficient improvement in one version may eliminate another version altogether.

The chain of **if** statements also is difficult to interface with any of the available techniques [24, 17] for calculating the performance of a program, particularly in light of the possibility of changing the **if** structure as the result of performance changes.

This paper proposes instead a scheme where all the versions of a particular computation are identified as candidates for binding to a generic name. We call this technique, *performance polymorphism* by analogy to the conventional polymorphism where different functions of the same name operate on different data types. We investigate the problem of binding in performance polymorphism, and the problem of how a program may allocate its available time and resources among performance-polymorphic subtasks.

The example we use is that of sorting on a parallel computer system. We have (at least) three different sorting techniques available: a fast insertion sort `isort`, a heap sort `hsort`, and a parallel merge sort `bsort`. The amount of time that these sorts take is $An^2 + Bn + C$, $Dn \log n + E$, and $F(n \log n)/p + G \log p + H$, respectively, where n is the number of elements to sort and p is the number of processing elements given to the parallel sort. We expect H, the overhead of starting the parallel sort, to be quite large. It may therefore be more effective to sort a short list of numbers on the local processor, without using the parallel sort. We also expect the constant factor in the time required for heap sort to be greater than that for insertion sort; it therefore may be more effective to choose the $O(n^2)$ insertion sort for extremely short lists.

To manage this information, a number of different items need to be taken into account. The programmer needs some way to specify the constraints on time and resources that are in effect, in order to determine when the choice of a version is feasible. In addition, a figure of merit for a given version may have to be defined, in order to choose the better of two feasible versions. The *binding* problem, the problem of determining the most appropriate version, must then be solved. Solving the binding problem demands information about the expected performance of versions of the program, and the system must therefore provide a capability of predicting the performance of a version. Finally, the situation where a single set of constraints governs the selection of different versions of several functions may arise. In this situation, we must solve the *allocation* problem, the problem of determining the most effective division of resources among the different functions.

The components necessary to solve these problems have all been discussed to some extent by other researchers. The concept of polymorphism is, of course, familiar from very high level languages such as ML [2]. Applying polymorphism to performance criteria is less well developed, although a less general system, where certain performance decisions were made through the use of 'program adaptations,' was described by Schwan [22]. More recently, Samples [21] has presented a system for selecting alternative implementations of an abstract data type. While his system does not address real-time systems specifically, its model of function substitution is similar to ours.

Samples also presents one way of providing the performance information of functions — a way which depends on the programmer's analysis of the algorithms. An alternative way, based on predicting at compile time the time and resources that a function will consume, has been explored by Mok [17], and Stoyenko [24] — Stoyenko's method, however, can make little use of the programmer's knowledge, while Mok's is awkward to use for many common program structures, such as nested loops. By contrast, our method can make use of both the program's observed performance and the programmer's knowledge about its theoretical behavior.

3 The FLEX language

The Concord project at the University of Illinois [15] is engaged in building a programming environment for large real-time systems, supporting explicit timing constraints [14, 12], imprecise computations [16], and multiple implementations of a single computation, to support different configuration and timing constraints. This environment is built around the programming language FLEX [19], a derivative of C++ [25].

The means in FLEX whereby the programmer specifies the limits on time and resources is the *constraint block*. Constraint blocks are discussed more thoroughly in other papers [14, 12]; we give only a short discussion here. A constraint block identifies a set of constraints on time and resources that must apply when a given section of the program is in execution. It takes the form:

constraint block [*label*:] (*constraint; constraint;* ...)
  [$\rightsquigarrow$ {...}] {
  ... *statements* ...
}

This block specifies that the sequence of statements is to be executed, and asserts that all of the constraints will be satisfied during the execution of the block. The optional label is provided so that one constraint block can refer to another. The block statements following the '$\rightsquigarrow$' is optional, and represents an exception procedure to be executed if any of the constraints fails.

A constraint may be either a Boolean expression (which is treated as an assertion to be maintained throughout the block's lifetime) or a timing constraint, which describes a constraint on the time at which the block may begin or end its execution.

Each block has *start* and *finish* times that refer to the time that the block's execution begins and ends. For convenience with periodic and sporadic jobs, a block also has an *interval,* which refers to the difference between the start times of two successive executions of the block, and a *duration,* which refers to the difference between the *start* and *finish* times of a single execution of the block. Figure 1 shows all four of these timing attributes.

Some examples of constraints that can appear at the head of a constraint block are:

Example 1 (duration ≤ 1500)

This declaration specifies that the computations, whenever they begin, must take no more that 1500 time units of real time to complete.

Example 2 (start $\geq t_1$; finish $\leq t_2$; temperature < 110)

This constraint specifies that the computations must start at a time no earlier than t_1, and end at a time no later than t_2.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

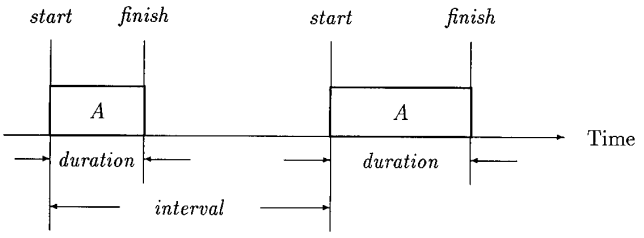


Figure 1: Timing attributes of constraint blocks

During the computations, the contents of the `Constrained` variable *temperature* must be less than 110 at all times. Presumably, this variable will be maintained by a process that polls a sensor; the process might look something like:

```
for (;;) {
    (interval >= dt; interval <= dt + epsilon) {
        temperature = read_sensor();
    }
}
```

Example 3 A : (start $\geq B$.finish)

This constraint specifies that the computation, identified as block A, may not begin until some other computation, identified as block B, has completed.

Example 4 B : (1)

This constraint imposes no conditions on the execution of the computation, and simply serves to identify it as block B, presumably so that another constraint block can refer to its start and stop times.

FLEX also provides the ability to measure the performance of a block of code. Any sequence of statements surrounded by '{' and '}' may be instrumented by a `#pragma measure` directive. This directive requests that the usage of a particular resource be measured, and provides a model for the amount of the resource that the programmer expects the block to consume. The parameters of the model are determined from the measured performance. Like constraint blocks, this facility is described more fully in another paper [13]; we merely give an illustrative example here.

Example 5 .

```
void hsort (int n, int *list)
#pragma measure mean duration \
              defining D, E in D*n*log(n) + E
{
    ...
}
```

This example shows the definition of a function that sorts an array of integers using heapsort. The variable n is the length of the array; the `measure` directive requests that the amount of real time used to do the sort be measured. The programmer expects that the amount of time will be described by a function of the form, $Dn \log n + E$, for some constants D and E. These constants will be determined from the data obtained from the measurement, and a measure of statistical confidence in their values will be reported.

4 The binding problem

In the previous section, we have seen how the FLEX language allows specifying the constraints on time and resources that must be met by a block of code, and the expected time and resource requirements for the block. This section introduces *performance polymorphic* functions, *i.e.,* multiple functions that carry out the same computation but differ in their performance characteristics.

Let the interface by which the computation is requested be a function $F(x_1, \ldots, x_n)$. The x_i's are the formal parameters accepted by the function. Let $g_1(x_1, \ldots, x_n)$, $g_2(x_1, \ldots, x_n)$, ... be the functions that are provided as implementations of F. We call each of these functions a *candidate.* Each candidate g_i will require some quantity of time and resources to carry out the computation. These quantities can depend on the arguments to the function; for example, a sort function requires an amount of time that is dependent on the length of the list of items to sort. For each different type of resource $R_1, \ldots, R_m$, we define *claim functions* $c_{i1}(x_1, \ldots, x_n), \ldots c_{im}(x_1, \ldots, x_n)$; each function c_{ij} describes the amount of resource R_j required by candidate g_i.

A candidate is *feasible* if its resource claims satisfy all constraints. If a candidate g_i is feasible, then it is possible to use g_i for a given invocation of the performance polymorphic function F. If no candidate is feasible, then the requested computation cannot be carried out and still meet all resource constraints; this situation results in a timing fault and invokes the recovery procedure of the failed constraint. It is also possible for more than one candidate to be feasible; the feasibility test alone does not uniquely determine a candidate.

To determine which of several feasible candidates is the best choice for a given invocation, we introduce an *objective,* or *figure of merit* M. M is an arithmetic expression involving the claims c_{ij}. The binding step, when the program invokes a performance polymorphic function F, is defined to be:

> **Binding step.** *Choose from among the feasible candidates g_i, g_j, ... the unique g_k that gives the maximum figure of merit M.*

4.1 Language support

In order to implement the model of performance polymorphism, we must augment FLEX with a means to describe the candidates, their performance, and the objectives. We begin by providing a means to declare a performance polymorphic function, by allowing the keyword `perf_poly` to replace the function body in a function declaration.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

Example 6 .

```
void sort (int n, int* list) perf_poly;
```

This example declares `sort` to be a performance polymorphic function accepting an integer `n` and a 'pointer to integer' `list`. Presumably, `sort` is a function that sorts a list of integers.

Next, we provide a way to give a candidate for a performance polymorphic function. We do this by adding a third type of external definition to the language:

external-definition:
    function-definition
    declaration
    provide-declaration
provide-declaration:
    `provide` *identifier* `for` *fct-declarator* ;

Example 7

```
provide hsort for void sort (int, int*);
```

This example supplies the function `hsort` as a candidate for the `sort` function defined above. We note that the types of the function's parameters, and the function's return type, must be repeated here. This repetition in necessary because the C++ language allows overloaded functions, *i.e.,* functions that have the same name and are selected according to their parameter or return types. The `provide` declaration has no context in which to infer the types, so they must be specified explicitly. For instance, there may be another `sort` function that sorts a list of floating-point numbers.

Finally, we need to add a description of the objective. We do this by extending the definition of a compound statement:

compound-statement:
    pragma-objective$_{opt}$ { *statement-list*$_{opt}$ }
pragma-objective:
    `#pragma objective` *direction expr direction:*
    `maximize`
    `minimize`

Example 8 .

```
#pragma objective minimize duration
{  ...  }
```

The directive shown in this example requests that the system minimize the real time taken to perform the computations enclosed in the braces, subject to all constraints.

We now have the declaration of performance polymorphic functions, of the candidates for a function, and of the constraints that apply to an invocation of a function. The only other data needed to perform the binding step are the claims that describe the resource requirements for the candidates.

The easiest way to provide these claims is to measure the performance of the candidate using `#pragma measure`, as described in Section 3. If this mechanism is used, the claims will be computed automatically. Another source for timing data is a timing analyzer that examines the code of a function and gives its resource claims, according to some model of the underlying hardware. Mok [17] and Stoyenko [24] provide systems that carry out this type of analysis.

Still another method is for the user to provide a non-parametric model that gives the resource claims directly. In this case, the user provides a function that returns the resource consumption as shown in the next example.

best-function := UNKNOWN;
best-merit := MINDOUBLE;
for $i := 1$ **to** n **do begin**
    calculate resource claims for candidate g_i;
    calculate feasibility based on claims;
    if *(candidate* g_i *is feasible)* **then begin**
        calculate figure of merit M *based on claims*
        if ($M >$ *best-merit*) **then begin**
            best-merit := M;
            best-function := g_i;
        end
    end
end;
if *best-function* = UNKNOWN **then**
    timing fault
else
    invoke best-function;

Figure 2: Algorithm for late binding at run time

Example 9 .

```
double _hsort_duration (int n, int* list)
{
  return 5.9e-7 * n * log(n) + 5.9e-6 * n + .009;
}
```

This somewhat contrived example declares that the running time for a call to `hsort` is $5.9\times10^{-7}n\log n+5.9\times10^{-6}n+.009$ seconds. The system determines that the function represents a performance measurement by the fact that it uses a reserved name of the form, `_x_duration`; words beginning with underscores are reserved to the implementor in C++, so this name cannot conflict with any function that the programmer would normally provide. Examples of similar performance functions can be found in Samples [21].

4.2 Run-time binding

Given the information described above, late binding at run time is easy to describe. The algorithm is shown in Figure 2. This algorithm simply loops through all the candidates, looking for feasibility, and selects the candidate with the greatest figure of merit. It is obviously somewhat expensive (the computation of the resource claims is a loop that is nested within the loop through the candidates), although it can be improved somewhat by caching the resource claims and feasibility information, assuming that a few configurations of the parameters account for most of the invocations.

The way that the caching would operate is to record the configuration of the parameters for the last few calls to the function, and as the first part of the binding step, check whether the parameters supplied match a recent invocation. If so, the binding need not be recomputed; the system can simply reuse the binding that was used on the earlier invocation. We have not experimented with this technique.

The advantage of run-time binding is its flexibility; it is required for cases where the relevant parameters cannot be determined at compile time. Such cases include data-dependent performance, where input data obtained at run time can affect the binding decision, fault-tolerant systems, where run-time binding may be needed to reconfigure around a system failure, and run-time allocation of time and resources, which is discussed more thoroughly in Section 5.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

The disadvantage of run-time binding is the overhead involved in making the binding decision; we therefore prefer early binding in the cases that do not require late binding. We also recommend appropriate choice of granularity for performance polymorphic functions that will require late binding. Short functions that complete in only a few instruction times are not appropriate candidates, as the cost of choosing an inappropriate function is small compared to the cost of binding.

4.3 Early binding

If all the candidates are known at compile time, then we can begin to consider how to reduce the strength of the binding operation. We begin by expanding the binding code in line, constructing a *can-bind* predicate for each of the candidate versions. Each *can-bind*(g_i) predicate represents the statement, "g_i is feasible, and g_i's figure of merit is the highest of all the feasible candidates." The result is a set of guarded commands [4]:

$$\begin{array}{ll} \textbf{if} & \textit{can-bind}(g_1) \rightarrow g_1(x_1, x_2, \ldots, x_a) \\ [] & \textit{can-bind}(g_2) \rightarrow g_2(x_1, x_2, \ldots, x_a) \\ & \vdots \\ [] & \textit{can-bind}(g_k) \rightarrow g_k(x_1, x_2, \ldots, x_a) \\ \textbf{fi} & \end{array}$$

We perform algebraic simplification on the predicates, and are then able to consider the following set of optimizations [11, 18]:

Constant propagation. Constants appearing in the guards may be propagated outwards. In particular, if all of the variables in a guard are constant, then the guard can be evaluated statically, giving compile-time binding, or at least elimination of one or more candidates at compile time.

Code motion. A subexpression in a guard that is constant for the span of a region of code may be moved outside that region. This motion may allow a binding step to be executed only once for a function invocation within a loop.

Common subexpression elimination. Several binding decisions may be reduced to one by combining subexpressions in the guards; this step is particularly useful as the generation of the *can-bind* predicates produces considerable redundancy.

Dead code elimination. If a guard is known to test false always, it can be eliminated from consideration.

Guard subsumption. If a candidate g_i is known to be feasible whenever another candidate g_j is feasible, and g_i's figure of merit is greater than g_j's for g_j's entire domain of feasibility, then g_j may be eliminated from consideration.

These steps reduce the choice of binding time (compile time, and early vs. late binding at run time) to a problem in data flow analysis that may be solved by a traditional optimizer in the compiler.

Table 1: Combining resource claims

Construct	Space-like	Time-like
a; b	$\max(c_a, c_b)$	$c_a + c_b$
loop a **end**	c_a	$n \cdot c_a$
if $(c) \rightarrow a$ **else** b	$\max(c_a, c_b, c_c)$	$\max(c_a, c_b) + c_c$
cobegin a; b **coend**	$c_a + c_b$	$\max(c_a, c_b)$

Note: n represents the number of times that the loop will be executed.

4.4 Notes on the binding strategy

The optimizations presented above are *backward propagation,* in that the binding decision is made late, but the optimizer factors portions of the decision and propagates them to earlier in the program. An alternative optimization strategy that was considered and rejected is *forward* propagation. In the latter strategy, each time that a variable on which performance depends is changed, all the binding decisions that relate to that variable are recomputed immediately.

Forward propagation has the potential for providing much more aggressive optimization, particularly in the case where most of the variables on which performance depends are system configuration parameters, and therefore are not often changed. In this case, once the configuration is determined, the binding decisions are made once, with no overhead at each function call.

It is difficult, though, to integrate forward propagation with the rest of the system. The variables on which performance depends are often presented as parameters to procedures, and effective forward propagation requires sophisticated interprocedural analysis. If such analysis is available, backward propagation can be made to perform almost as well.

In any case, we foresee that the usual use of performance polymorphism will be coarse-grained; the cost of executing one of the functions will be much greater than the cost of making the binding decision. In such uses, the optimization issues are largely irrelevant, as the cost of binding will not be a significant factor of the cost of running the entire system.

5 The allocation problem

So far, we have considered only the case where there is a single performance polymorphic function to be bound. If two or more functions are to be bound, we have an *allocation* problem: we must decide how most effectively to divide the available time and resources among the multiple functions, together with deciding which of the candidate functions to choose.

This problem is a difficult one in general; a wide class of allocation and scheduling problems are NP-complete [6, 7, 10]. Fortunately, many actual resource allocation problems are computationally tractable; to see why this is so, let us first examine the ways that resource claims are combined by programming constructs.

The major resource classes are called *time-like* and *space-like* resources. A time-like resource, in general, models some sort of time, or another resource that, once consumed, is never reused. By contrast, a space-like resource models a physical object; once a task has completed using an object, another task can use it. Space-like resources include memory, disc space, processing elements, critical sections, and so on.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

In order to combine the resource claims of two operations that use time-like and space-like resources, the only operations that we need consider are summation, taking a maximum, and multiplication by a constant factor. Table 1 shows how the two types of resource are combined under common programming language constructs. For example, consider the **if** statement:

$$\textbf{if } (c) \rightarrow a \textbf{ else } b \ .$$

The amount of time needed to execute this statement in the worst case is the worst-case amount of time needed to evaluate the condition c, plus the greater of the worst-case amounts of time needed to execute the two parts a and b. The amount of dynamically allocated storage is simply the greatest amount of storage required to execute a or b, or to evaluate the condition c.

5.1 The knapsack-sharing problem

In these resource sharing problems, there is usually a single resource that forms the bottleneck, or limiting factor. For instance, the objective may translate to 'use the minimum amount of memory possible, subject to the constraint that the computation must meet its deadline,' or 'return results that are as precise as possible, subject to the constraint that all deadlines are met.' If we look at the table describing how resources are to be combined, we see that the constraint that all deadlines are met takes the form

$$\sum_i w_i t_i \leq F \ ,$$

where the t_i's are the times taken by individual operations and the w_i's are constant weight factors associated with these times. The natural interpretation of the weights w_i is that each w_i represents the expected number of times that operation i is performed during the task. This number can be obtained from loop analysis or from program profiling [21].

Similarly, the constraint to minimize space or maximize precision takes the form:

$$\text{maximize } \min_i h_i(t_i) \ .$$

Each function $h_i(t_i)$ represents the amount of the space-like resource that is needed to complete the subtask i in an amount of time no greater than t_i.

This is exactly the framework of the *knapsack sharing problem*, which has been studied extensively by Brown [1]. He presents an algorithm for solving this problem in linear time, for any piecewise continuous tradeoff functions h_i.

The algorithm begins by making the substitution $x_i = w_i t_i$, and $\varphi_i(x_i) = h_i(x_i/w_i)$. This substitution eliminates the weights, normalizing the problem to:

$$\text{maximize } \min_i \varphi_i(x_i) \text{ subject to } \sum_i x_i \leq F.$$

We also let $L(\Phi) = \sum_i \varphi_i^{-1}(\Phi)$.

The algorithm then is:

Step 1. Set R to the set of all tasks i.

Step 2. Let τ be the task in R with the largest $\varphi_\tau(0)$. If $L(\varphi_\tau(0)) \leq F$, go to Step 3. Otherwise, remove τ from R and repeat this step.

Step 3. For all the tasks t that have been removed from R, set $x_t = 0$. For all the remaining tasks, solve $L(\Phi) = F$ for Φ, and then set $x_i = \varphi_i^{-1}(\Phi)$.

The interested reader is referred to [1] for a proof of the algorithm, and another algorithm that works even if the time is constrained to be an integer.

The only difficult step in the algorithm is inverting the function $L(\Phi)$. We may assume, however, that L is nondecreasing—applying more resources never makes a task take more time—and therefore find the inverse for the single value F by some simple and fast method like bisection.

As an illustration, let us consider the problem of sorting several lists of numbers, one after the other, on a parallel machine. We have the constraint that all the sorts must be completed in a given time, and the objective of minimizing the number of processing elements required. The task i is the task of sorting the ith list. We control the number of processing elements by subjecting the entire set of tasks to a constraint of the form:

$$(\mathit{proc_elts} \leq p) \ ,$$

and impose deadlines on the individual tasks by making each subject to a constraint

$$(\mathit{finish} \leq d_i) \ .$$

The problem is to determine the number of processors p and the deadlines d_i.

The knapsack-sharing solution to the problem is shown graphically in Figure 3. The ordinate represents the finish times of the subtasks. Note that these times are absolute completion times; the curve for task 2 includes task 1's timing requirement. The abscissa represents the number of processors allocated; allocating more processors reduces the time needed to complete the tasks.

Using the deadline for the composite task (shown as the uppermost horizontal line in the figure), the knapsack-sharing algorithm finds the value of p needed to complete the whole job before the deadline (the vertical line). It commits to allocating p processors. Given p, it then finds appropriate deadlines d_i to set for the subtasks by finding how rapidly they can be completed on p processors.

Unfortunately, a problem like, "minimize the time required for the task subject to the constraint that its memory size not exceed n" is not nearly as simple; it becomes a combinatorial bin-packing problem rather than a knapsack sharing problem. For these sorts of problems, we will likely have to content ourselves with either scheduling techniques for a specific problem, or else techniques that return only an approximate optimum. These techniques include simulated annealing[3], Lagrange relaxation [5], and various sorts of heuristics like 'first fit' and 'best fit' [10].

Since the allocation method is likely to be application dependent, we do not provide a single allocator in the system. Instead, we allow the programmer to provide the allocation function, by using nested constraint blocks. The code for a typical case resembles Figure 4. In this way the allocation method is brought out explicitly, and the implementor can experiment with different techniques without needing to modify the runtime library or operating system.

[3] A good survey of this technique is to be found in section 10.9 of [20].

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

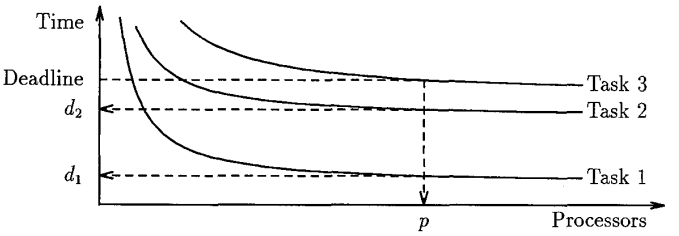


Figure 3: Example of knapsack-sharing allocation.

```
{
    (Decide, based on the time and resources available, the
    amount of time to allocate to the two subordinate tasks.
    Place the two amounts of time in variables A and B.)
    constraint-block (duration ≤ A) {
        ⋮
    }
    constraint-block (duration ≤ B) {
        ⋮
    }
}
```

Figure 4: A typical block with allocation

6 Final Notes

6.1 Experience

We have implemented the system that this paper describes, and are using it to build trial systems and gather further information about its performance. The overall architecture is shown in Figure 5.

The FLEX compiler is based on the GNU C++ compiler [23, 26], which was chosen because its source code is readily available, and because it runs on a wide variety of machines, facilitating portability. The compiler, in addition to producing an object program, also produces files that describe the programmer's model for program performance (as obtained from the `#pragma measure` directives).

The executable program writes a file that contains the observed performance data for the `#pragma measure` directives. A separate program, the 'dynamic timing analyzer,' reads these data and the performance model and determines the parameters that give the best fit. These parameters are recorded in a file that is read by the executable program at initialization time, giving it the information needed to do more accurate binding and allocation.

The portions of the system relating to 'static timing analysis' are not yet implemented, but are planned as a future enhancement. Static timing analysis refers to determining the performance of a program by examining it at the instruction level and using some model of the underlying hardware to deduce its performance [24, 17], rather than resorting to measurement of the actual performance. Some version of this type of analysis will be required to perform compile-time binding of performance polymorphic functions, as compile-time binding requires complete performance information, even about sections of the program that are not measured for dynamic analysis.

Despite the fact that we do run-time binding only, we find that for medium to coarse-grained problems, the overhead of the binding of performance polymorphic functions is negligible, adding perhaps a few hundred microseconds to the execution time of a function that takes from hundreds of milliseconds to seconds. We find that using performance polymorphism for smaller functions is ill advised, but have found few if any functions that are naturally substituted at that fine a level.

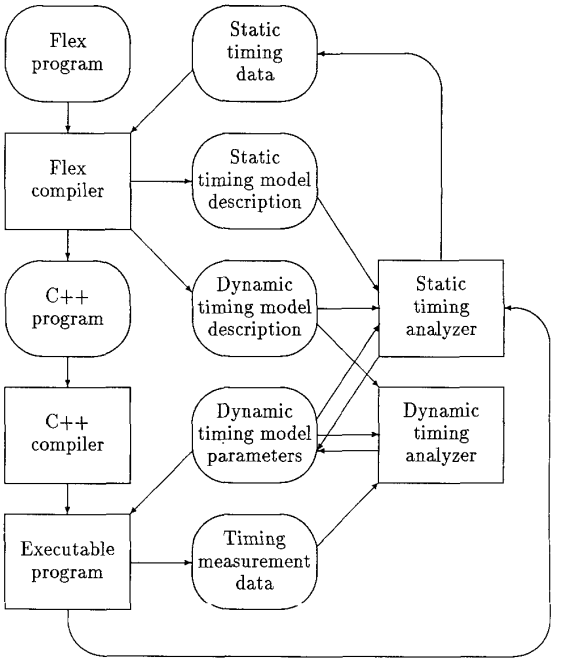


Figure 5: System supporting performance polymorphism

As we were building the code for the subsystem that analyzes the performance data, we found performance polymorphism to be a natural model for selecting an algorithm to determine the singular value decomposition of a matrix $A = U\Sigma V^T$, where $UU^T = V^TV = I$ and Σ is diagonal. There are two principal algorithms used for the problem today: one due to Golub and Reinsch [8] and another due to Chan [3]. Golub [9], in a *tour de force* of analysis, gives the figures shown in Table 2 as a measure of the relative amount of work each algorithm does. The expressions in the table are the "flop counts" familiar to numerical analysts.

To compare this analysis with that of performance poly-

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

Table 2: Comparison of SVD algorithms
(after Golub[9, page 175])

Required data	Golub-Reinsch algorithm	Chan algorithm
Σ	$2mn^2 - \frac{2}{3}n^3$	$mn^2 + n^3$
Σ, V	$2mn^2 + 4n^3$	$mn^2 + \frac{17}{3}n^3$
Σ, U_1	$7mn^2 - n^3$	$3mn^2 + \frac{16}{3}n^3$
Σ, U_1, V	$7mn^2 + \frac{11}{3}n^3$	$3mn^2 + 10n^3$

	Table 2 Method					Measurement Analysis				
n	2	6	10	14	18	2	6	10	14	18
m										
2	·					·				
6	●	·				·	·			
10	●	●	·			·	·	·		
14	●	·	·	·		·	·	·	·	
18	●	·	●	·	·	·	·	·	·	·
22	●	·	●	·	·	·	·	·	·	·
26	●	·	●	●	·	o	·	·	·	·
30	·	·	·	●	●	o	·	·	·	·
34	●	·	·	●	●	·	·	·	·	·
38	·	·	·	●	●	o	·	·	·	·
42	·	·	·	●	●	o	·	·	●	·
46	·	·	·	·	●	·	·	·	·	·

Solid circles — Chan algorithm chosen incorrectly
Open circles — Golub-Reinsch algorithm chosen incorrectly
Small dots — Correct algorithm chosen.

Figure 6: Comparison of algorithm choices

morphism, we constructed performance models for both algorithms, using cubic functions of m and n for the times. The fact that the expected times were cubic was derived by simple inspection of the loop structure of the program.

We then ran a measurement of the time required to factor a number of matrices of various sizes, in order to determine the model parameters. A performance-polymorphic function was then set up, with the Golub-Reinsch and Chan algorithms as its two candidate functions.

The run time required for the performance polymorphic function was compared with that required for the function chosen by the method of Table 2. The results are summarized in Figure 6. We note that the decision based on flop counts favors the Chan algorithm, choosing it (black circles) in many cases where the Colub-Reinsch algorithm would, in fact, be faster. The decision based on observed performance, by contrast, makes only a few incorrect choices. The improved decision is borne out on the run times taken; the test system took 140.9 seconds of machine time on a Sun 3/50 to factor the sample matrices when the rule from Table 2 was used, and this improved to 132.6 seconds when the performance polymorphic function was used.

In this example we see that using performance polymorphism not only made the programmer's job easier (little work was required in analyzing the algorithms), and still led to better performance than a program that depended on a detailed analysis of the run times of the two alternatives.

6.2 Conclusions

In addressing the problems of hard real-time systems, we often come upon problems where we must decide among qualitatively different actions based on quantitative information about resource constraints. Multiple versions of a function, having a homogeneous interface but meeting different constraints, may be present. Performance polymorphism is a unified model to express the choice among these multiple versions in a way that is both natural and powerful. It allows the flexibility of adding new versions at any time, of adapting to unforeseen constraints, and of adapting to automatically generated variants of a procedure (as, for example, might come from a parallelizing compiler).

We have developed a means to implement the model of performance polymorphism that requires very low overheads at run time, by analyzing the constraints and propagating performance information backwards in order to bind performance polymorphic functions as early as possible. We have implemented a version of the tools required for the technique, and are gaining experience with their use in actual practice.

While there are, of course, other ways to view the problem of variant functions under performance constraints, we are confident that our method is feasible, powerful, and natural for the programmer.

References

[1] J. Randall Brown. The knapsack sharing problem. *Operations Research*, 27(2):341–355, April 1979.

[2] Luca Cardelli and Peter Wegner. Understanding types, data abstractions, and polymorphism. *ACM Computing Surveys*, 17(4):471–522, December 1985.

[3] Tony F. Chan. An improved algorithm for computing the singular value decomposition. *ACM Transactions on Mathematical Software*, 8(1):72–83, January 1982.

[4] Edsger W. Dijkstra. *A Discipline of Programming.* Prentice-Hall, Englewood Cliffs, N. J., 1976.

[5] Hugh Everett. Generalized Lagrange multiplier method for solving problems of optimum allocation of resources. *Operations Research*, 11:399–417, 1963.

[6] M. R. Garey, R. L. Graham, D. S. Johnson, and A. C.-C. Yao. Resource constrained scheduling as generalized bin packing. *Journal of Combinatorial Theory*, 21:257–298, 1976.

[7] M. R. Garey and D. S. Johnson. Complexity results for multiprocessor scheduling under resource constraints. *SIAM Journal of Computing*, 4, 1974.

[8] Gene. H. Golub and C. Reinsch. Singular value decomposition and least squares solutions. *Numerische Mathematik*, 14:403–420, 1970.

[9] Gene H. Golub and Charles F. Van Loan. *Matrix Computations.* Johns Hopkins, Baltimore, Maryland, 1983.

[10] D. S. Johnson, A. Demers, J. D. Ullman, A. R. Garey, and R. L. Graham. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal of Computing*, 3:299–325, 1974.

[11] Neil D. Jones and Steven S. Muchnick. Binding time optimization in programming languages: some thoughts toward the design of an ideal language. In *Proceedings of the Third Symposium on Principles of Programming Languages*, pages 77–94, ACM, Atlanta, Ga., January 1976.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.

[12] Kevin B. Kenny and Kwei–Jay Lin. *Implementing Timing Constraints in* FLEX. Technical Report UIUCDCS-R-90-1567, University of Illinois at Urbana–Champaign, January 1990.

[13] Kevin B. Kenny and Kwei–Jay Lin. *A Measurement-based Performance Analyzer for Real-Time Programs.* Technical Report UIUCDCS-R-90-1606, University of Illinois, June 1990.

[14] Kwei–Jay Lin and Swaminathan Natarajan. Expressing and maintaining timing constraints in FLEX. In *Proceedings of the Ninth Real-Time Systems Symposium*, pages 96–105, Huntsville, Ala., December 1988.

[15] Kwei–Jay Lin, Swaminathan Natarajan, and Jane W.–S. Liu. Concord: a system of imprecise computations. In *Proceedinngs of COMPSAC '87*, IEEE, Tokyo, Japan, October 1987.

[16] Kwei–Jay Lin, Swaminathan Natarajan, and Jane W.-S. Liu. Imprecise results: utilizing partial computations in real-time systems. In *Proceedings of the Eighth Real-Time Systems Symposium*, pages 210–217, San Jose, Calif., December 1987.

[17] Aloysius K. Mok, Prasanna Amerasinghe, Moyer Chen, and Kamtorn Tantisirivat. Evaluating tight execution time bounds of programs by annotations. In *Proceedings of the Sixth IEEE Workshop on Real-Time Operating Systems and Software*, pages 272–279, IEEE, May 1989.

[18] Steven S. Muchnick and Neil D. Jones. *Program Flow Analysis: Theory and Applications.* Prentice–Hall, Englewood Cliffs, New Jersey, 1981.

[19] Swaminathan Natarajan and Kwei–Jay Lin. FLEX: toward flexible real-time programs. In *Proceedings of the International Conference on Computer Languages*, pages 272–279, Miami, Fla., October 1988.

[20] William H. Press, Brian P. Flannery, Saul A Teukolsky, and William T. Vetterling. *Numerical Recipes.* Cambridge University Press, 1986.

[21] A. Dain Samples and Paul N. Hilfinger. *Profile-Driven Compilation.* Technical Report, University of California at Berkeley, February 1990. To appear in *Proc. Intl. Conf. on Information Technology,* Tokyo, October 1990.

[22] Karsten Schwan, Rajiv Ramnath, Sridhar Vasudevan, and David Ogle. A language and system for the construction and tining of parallel programs. *IEEE Transactions on Software Engineering*, 14(4):455–471, April 1988.

[23] Richard M. Stallman. *Using and Porting GNU CC.* Free Software Foundation, Cambridge, Massachusetts, 1989.

[24] A. D. Stoyenko. A schedulability analyzer for real-time Euclid. In *Proceedings of the Eighth Real-Time Systems Symposium*, pages 218–227, San Jose, Calif., December 1987.

[25] Bjarne Stroustrup. *The C++ Programming Language.* Addison-Wesley, Reading, Massachusetts, 1986.

[26] Michael D. Tiemann. *Users' Guide to GNU C++.* Free Software Foundation, Cambridge, Massachusetts, 1989.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 01:44:43 UTC from IEEE Xplore. Restrictions apply.